

Laboratoire de High Performance Coding semestre printemps 2026

High Performance Coding (HPC) - Labo 4 SIMD

Temps à disposition : 4 périodes (2 séances de laboratoire)

Récupération du laboratoire dans l'archive sur Cyberlearn

1 Objectifs de ce laboratoire

Dans ce laboratoire, nous allons vous guider à travers le monde du SIMD. Le but est de vous familiariser avec ce concept et de comprendre, de manière simple, comment le SIMD permet d'accélérer un code, ainsi que la logique de son utilisation.

Dans un premier temps, nous vous inviterons à observer un code existant. Dans un second temps, vous devrez vous approprier un code simple afin de l'accélérer **sans l'aide du compilateur**.

Vous trouverez toutes les informations nécessaires concernant les instructions SIMD pour les processeurs Intel à l'adresse suivante : <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html>

2 Analyse de code

Dans cette première partie, vous allez observer un programme simple qui met à jour la position d'un grand nombre de points au cours de plusieurs itérations. L'objectif est d'étudier comment le compilateur peut **auto-vectoriser** une boucle et quel impact cela peut avoir sur le temps d'exécution.

2.1 Connaissances minimales

Les instructions SIMD permettent de traiter plusieurs éléments en parallèle au sein d'un *seul* cœur, grâce à des registres vectoriels :

- **SSE** : registres de 128 bits ;
- **AVX / AVX2** : registres de 256 bits ;
- **AVX-512** : registres de 512 bits.

Par exemple, en simple précision (`float`, 32 bits), un registre :

- SSE (128 bits) peut contenir $128/32 = 4$ floats ;
- AVX (256 bits) peut contenir $256/32 = 8$ floats.

Ainsi, si une boucle est vectorisée, elle peut potentiellement traiter **4 ou 8 éléments par itération** au lieu d'un seul.

3 Auto-vectorisation et organisation des données

Dans cette section, nous allons étudier comment le compilateur peut exploiter les instructions SIMD **sans modifier le code source**, uniquement à l'aide d'options de compilation.

L'objectif est de comprendre :

- comment le compilateur vectorise automatiquement une boucle ;
- quels sont les freins à la vectorisation ;
- pourquoi l'organisation des données en mémoire est cruciale pour les performances.

Nous travaillerons sur une boucle mettant à jour la position d'un ensemble de points au cours de plusieurs itérations. L'exécutable s'utilise de la manière suivante : `./simd_demo <nb_points> <nb_iterations>`.

3.1 Outils et méthodes d'analyse de code

Afin de faciliter l'analyse, nous allons introduire un outil d'analyse de code. Vous connaissez normalement `objdump`, mais celui-ci peut être difficile à lire et particulièrement fastidieux.

Une bonne alternative est l'outil `perf`, et plus particulièrement la commande `annotate`, qui permet d'annoter directement le code assembleur.

Pour pouvoir utiliser `perf annotate`, il faut d'abord enregistrer l'exécution du programme :

```
perf record <executable> <parametres>
```

Afin de vous simplifier la tâche, une mesure du temps d'exécution est déjà présente dans le code, autour de la zone d'intérêt. Cette zone a été identifiée par lecture du code et confirmée à l'aide de `perf` : en effet, `perf` permet d'identifier les zones où le code passe le plus de temps (en pourcentage de cycles d'exécution), et l'on observe clairement que la double boucle est la partie dominante.

Note : l'option `-g` présente dans le CMake est nécessaire. Sans elle, le code désassemblé n'est pas lisible. Cette option active les symboles de debug et rend généralement le code légèrement plus lent.

3.2 Première étape – Prise en main

Sans modifier ni le code ni le CMake, nous vous invitons à tester `perf` et à observer le code assembleur afin de vous en imprégner.

Notez également le temps d'exécution : il servira de **baseline** de comparaison pour les étapes suivantes du laboratoire.

3.3 Seconde étape – Optimisation par le compilateur

Nous allons maintenant nous intéresser à ce que le compilateur est capable de faire. Dans le laboratoire précédent, vous avez déjà exploré certaines optimisations.

Ici, nous nous concentrons sur les options globales `-O`. Ces options activent des optimisations automatiques du compilateur : réorganisation du code, simplification, suppression de calculs inutiles, etc., tout en conservant le même comportement observable du programme.

3.3.1 Optimisation niveau 2

Modifiez le niveau d'optimisation du CMake en `-O2`, recompilez le projet, puis observez avec `perf` ce qui a changé.

On remarque dans le code désassemblé l'apparition de vectorisation, absente auparavant. Comment la repérer ? On observe des instructions utilisant des registres `xmm`, qui sont des registres de 128 bits, capables de contenir 4 valeurs de 32 bits.

Le nom d'une instruction SIMD décrit ce qu'elle fait et sur quoi elle agit. Il peut être lu comme un mot composé :

`<operation><mode><type de donnees>`

1. **Opération** : partie la plus intuitive (add, mul, etc.).
2. **Mode** :
 - `s` pour scalar : une seule valeur est traitée ;
 - `p` pour packed : plusieurs valeurs sont traitées en parallèle.
3. **Type de données** :
 - `b` : entier 8 bits ;
 - `w` : entier 16 bits ;
 - `s` : float 32 bits ou entier 32 bits ;
 - `d` : double 64 bits ou entier 64 bits.

(Selon l'instruction, certaines combinaisons peuvent varier.)

Avec ces informations, pouvez-vous expliquer pourquoi le code est plus rapide avec `-O2` ?

3.3.2 Optimisation niveau 3

Modifiez à nouveau le CMake pour passer en `-O3`. Observez à nouveau le code désassemblé à l'aide de `perf`.

Qu'observez-vous de significativement différent par rapport à l'étape précédente ?

3.3.3 Optimisation pour un processeur cible

Les options `-O2` et `-O3` produisent du code générique, exécutable sur une large gamme de processeurs. Cependant, il est possible de cibler un processeur spécifique grâce à l'option `-march=`.

Deux possibilités s'offrent à vous :

1. `native` : le compilateur détecte automatiquement le processeur et active toutes les instructions qu'il supporte. **Attention : l'exécutable n'est alors plus portable.**
2. `<nom_processeur>` : permet de spécifier manuellement une micro-architecture (par exemple `haswell`).

Pour cette étape, comme le code n'est pas destiné à être partagé, ajoutez directement l'option `-march=native` après `-O3` dans le CMake.

Selon votre processeur, vous devriez voir apparaître de nouveaux registres :

- `ymm` : registres 256 bits ;
- `zmm` : registres 512 bits.

Pouvez-vous expliquer ce qui s'est passé et pourquoi le code est encore plus rapide ?

3.4 Optimisation du code

Jusqu'à présent, nous avons observé qu'en modifiant uniquement les options du compilateur, il est possible d'accélérer significativement l'exécution d'un programme.

Cependant, le compilateur ne peut pas modifier la logique du code : une mauvaise organisation des données limite fortement les performances.

Dans notre code, nous utilisons une structure de type *Array of Structures (AoS)* :

```
typedef struct {  
    float position;  
    float velocity;  
} Point;
```

Que cela implique-t-il ?

Si l'on instancie un tableau de 4 éléments, la mémoire est organisée ainsi :

```
[position] [velocity] [position] [velocity] [position] [velocity] [position] [velocity]
```

Et nous souhaitons effectuer le calcul suivant :

```
position += velocity * Delta
```

Les données ne sont pas organisées par type mais entremêlées. Il est donc impossible de charger directement un vecteur SIMD contenant uniquement des positions ou uniquement des vitesses.

Le compilateur doit charger des vecteurs mixtes, puis effectuer des opérations de *shuffle* ou *permute* pour séparer les données, avant de pouvoir effectuer les calculs vectorisés. Ces opérations supplémentaires sont visibles dans le code désassemblé et ont un coût en performance.

3.4.1 Aggraver le cas

Imaginons maintenant que l'on aggrave la situation en modifiant la structure :

```
typedef struct {  
    float position;  
    float velocity;  
    float otherNoisyValues[16];  
} Point;
```

Pouvez-vous expliquer pourquoi cette organisation est encore pire pour la vectorisation ?

3.4.2 Structure of Arrays (SoA)

Transformez la structure en *Structure of Arrays (SoA)* :

```
typedef struct {  
    float *position;  
    float *velocity;  
} Point;
```

Adaptez ensuite le code :

1. Allocation mémoire des tableaux ;
2. Boucle d'initialisation ;
3. Boucle de calcul ;
4. Libération de la mémoire.

Expliquez pourquoi cette organisation est plus favorable à la vectorisation et quelles en sont les limites.

Un problème subsiste : que se passe-t-il si la taille des données n'est pas un multiple de 8 ou 16 ? Les éléments restants doivent être traités séparément, ce qui entraîne des contrôles supplémentaires et des branchements.

3.4.3 Hybrid AoS vs SoA avec alignement

Une approche hybride entre AoS et SoA est possible. Elle consiste à créer des blocs de taille optimale et à les aligner par rapport au cache L1 lors de l'allocation mémoire.

Pour vous aider, utilisez plutôt que `malloc` :

```
posix_memalign(void **memptr, size_t alignment, size_t size)
```

Transformez la structure AoS en une version hybride avec une taille de bloc adaptée à votre architecture ainsi qu'un alignement cohérent :

```
#define ALIGNMENT <taille du cache L1>
#define BLOCK_SIZE <votre taille de bloc>
typedef struct {
    float position[BLOCK_SIZE];
    float velocity[BLOCK_SIZE];
} Point;
```

Adaptez le code :

1. Calcul du nombre de blocs ;
2. Allocation de la mémoire ;
3. Boucle d'initialisation ;
4. Boucle de calcul (incluant l'itération sur les blocs) ;
5. Libération de la mémoire.

Observez-vous un gain ?

4 À vous de jouer !

Dans `src/exercice.c`, vous trouverez un code qui traite un ensemble de samples. Le but est de pousser votre réflexion et d'optimiser ce code au maximum.

Une fonction `process_samples_simd` est fournie vide : vous êtes libre d'y implémenter toute solution SIMD de votre choix. Les résultats doivent toutefois être strictement identiques à ceux de l'exécution séquentielle.

Vous pouvez modifier les paramètres, créer de nouvelles structures ou variables si cela vous semble pertinent.

Pour vous aider, reportez-vous à la documentation mentionnée dans l'introduction.

5 Travail à rendre

Rédigez un rapport présentant votre analyse et votre compréhension de la première partie sur l'auto-vectorisation. Discutez de vos résultats et argumentez-les.

Dans un second temps, expliquez votre démarche et vos choix pour l'optimisation manuelle du code dans la seconde partie.